SQL Injection — Bypass del filtro de categorías mediante una condición lógica

1. Título

SQL Injection (SQLi) — Manipulación de una consulta SQL mediante el parámetro category

2. Descripción

Se identificó una vulnerabilidad de SQL Injection en el mecanismo de filtrado de categorías de una aplicación web.

La aplicación utiliza el parámetro `category` para filtrar los productos mostrados. Al enviar una comilla simple (`'`) en el valor del parámetro, el servidor respondió con un error HTTP `500 Internal Server Error`, lo que indicó un posible problema en el procesamiento de la consulta.

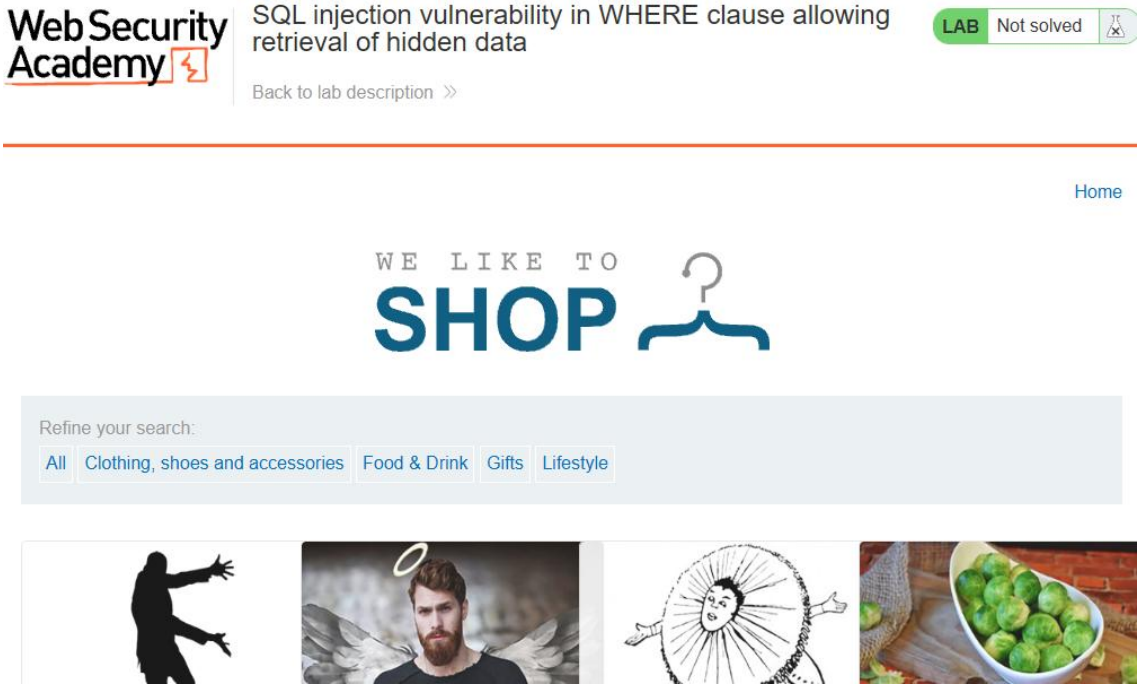
Posteriormente, se modificó el valor del parámetro para incorporar la expresión `' OR 1=1--`. La condición lógica siempre verdadera permitió alterar el comportamiento del filtro y recuperar productos que normalmente no se mostrarían con la categoría seleccionada.

El laboratorio de PortSwigger Web Security Academy confirmó la explotación exitosa mediante el estado **Solved**.

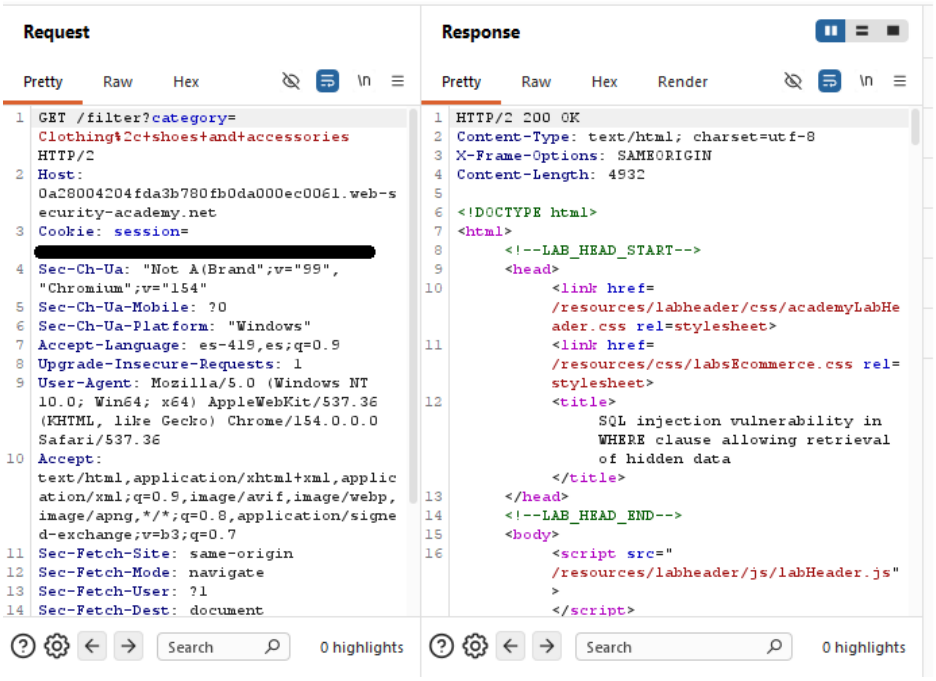
3. Evidencia

Se recopilaron cinco capturas de pantalla que documentan el proceso:

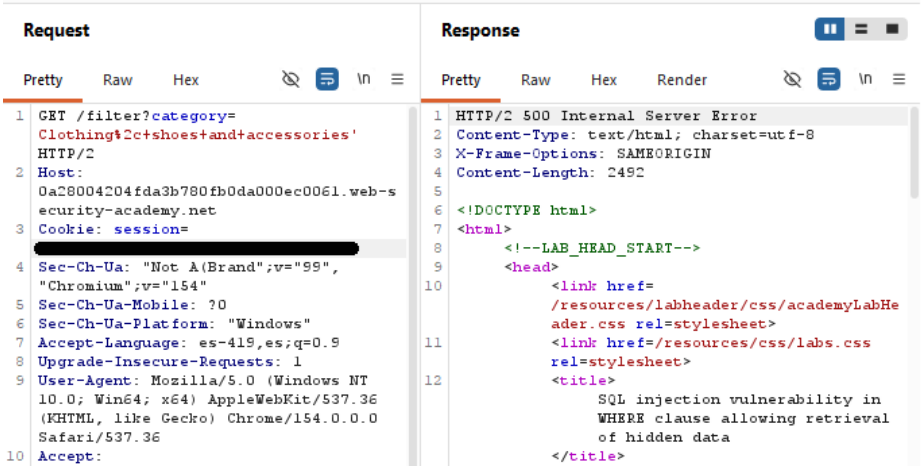
1. **Página inicial:** estado inicial de la aplicación web.



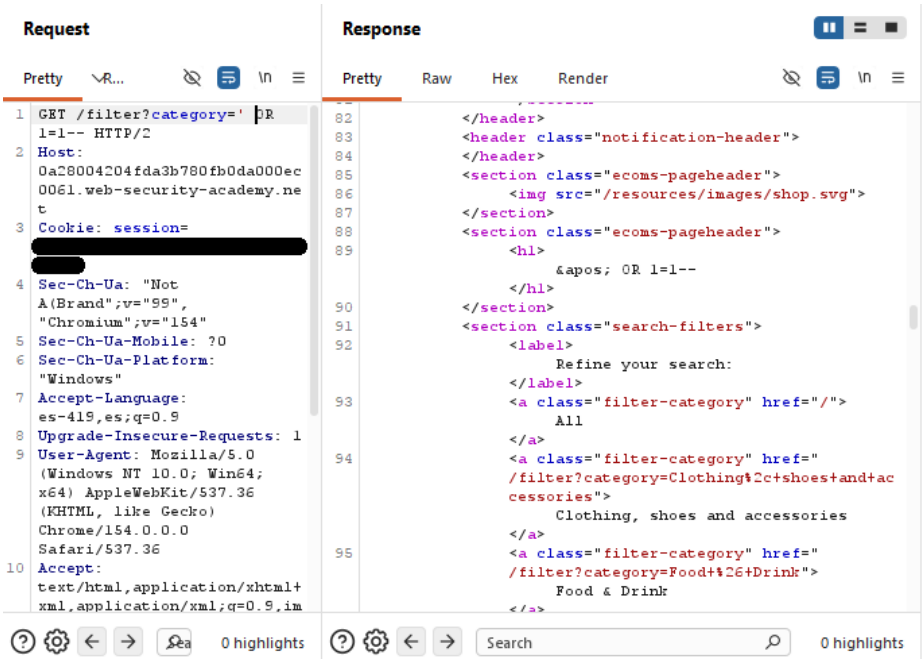
2. **Petición original:** solicitud HTTP al endpoint `/filter` con el parámetro `category` y su valor original.



3. **Error provocado por una comilla:** respuesta HTTP `500 Internal Server Error` tras modificar el parámetro.



4. **Prueba de SQL Injection:** petición modificada con la expresión `' OR 1=1--` y respuesta HTTP `200 OK`, junto con el contenido devuelto por la aplicación.



5. **Laboratorio resuelto:** confirmación de finalización exitosa del laboratorio.



Estas evidencias permiten documentar el comportamiento original, la señal inicial de error y la explotación confirmada.

4. Impacto (CIA)

Confidencialidad:

La vulnerabilidad permite recuperar datos que deberían estar restringidos por el filtro de categorías. Según los permisos de la base de datos y la consulta afectada, una explotación más amplia podría exponer información adicional.

Integridad:

No se demostró modificación, inserción ni eliminación de datos. Sin embargo, otras variantes de SQL Injection podrían afectar la integridad si la aplicación y los permisos de la base de datos lo permiten.

Disponibilidad:

No se comprobó una afectación persistente de la disponibilidad. La respuesta HTTP `500` observada al introducir la comilla simple indica que la entrada provocó un error durante el procesamiento de la solicitud.

5. Causa raíz

La causa raíz es la construcción o ejecución insegura de una consulta SQL que incorpora el valor del parámetro `category` sin separar adecuadamente los datos proporcionados por el usuario de la lógica de la consulta.

Esto permite que una entrada manipulada altere la condición de filtrado y cambie los resultados devueltos por la base de datos.

La validación de entradas por sí sola no constituye una defensa suficiente frente a SQL Injection.

6. Remediación

- **Utilizar consultas parametrizadas:** emplear consultas preparadas (*prepared statements*) para separar los datos de la estructura SQL.
- **Evitar la concatenación de entradas:** no incorporar directamente parámetros controlados por el usuario en cadenas SQL.
- **Aplicar el principio de mínimo privilegio:** limitar los permisos de la cuenta utilizada por la aplicación para acceder a la base de datos.
- **Manejar los errores de forma segura:** evitar exponer errores internos de SQL al cliente y registrar los detalles técnicos en los logs del servidor.
- **Realizar pruebas de seguridad:** verificar que las entradas manipuladas no puedan modificar la lógica de las consultas.

7. CVSS aproximado

CVSS v3.1: 6.5 — Medium (Medio), estimación provisional.

Vector orientativo: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

Esta estimación representa un escenario limitado en el que un atacante remoto sin privilegios puede recuperar información de baja sensibilidad. La puntuación definitiva depende de la sensibilidad de los datos expuestos, el alcance de la consulta vulnerable y las condiciones reales de explotación.

El laboratorio demuestra la alteración del filtro de productos, pero no acredita por sí solo acceso a información sensible ni modificación de datos en un entorno real.

8. Prioridad

Media — corregir antes de desplegar en producción.

La vulnerabilidad permite alterar una consulta SQL mediante un parámetro controlado por el cliente. Se recomienda priorizar la implementación de consultas parametrizadas y verificar que el filtro de categorías no pueda manipularse para acceder a información fuera de los resultados previstos.